

Hide-and-Seek Game Solver

Using Linear Programming & the Simplex Method

A Technical Report

Covering Game Theory Concepts, Code Explanation, and Test Cases

Team Members

Name	ID
Belal Mohamed Hussien	23010341
Ahmed Shrief	23010197
Moaz Hassan	23010858

May 6, 2026

Contents

1	Introduction	2
2	Conceptual Background	2
2.1	Two-Player Zero-Sum Games	2
2.2	Payoff Matrix Construction	2
3	Code Explanation	3
3.1	<code>solver.py</code> — HideAndSeekGame Class	3
3.1.1	<code>__init__</code> and <code>initialize_solver</code>	3
3.1.2	<code>play_game</code> — Single-Round Resolution	3
3.1.3	<code>simulate_game</code>	4
3.1.4	<code>solve_payoff_matrix</code> — LP via CPLEX	4
3.2	<code>main.py</code> — Flask REST API	5
3.2.1	Endpoints	5
3.2.2	Global State Management	6
3.2.3	Role Propagation	6

1 Introduction

This report documents a two-player, zero-sum Hide-and-Seek game solved via linear programming (LP). The system consists of two components:

- `solver.py` — defines the game, builds the payoff matrix, simulates play, and solves for Nash equilibrium strategies using IBM CPLEX.
- `main.py` — a Flask REST API that exposes the solver to a frontend client, maintaining persistent game state across requests.

The key mathematical tools are *zero-sum game theory* and the *Minimax Theorem*, solved numerically through LP duality.

2 Conceptual Background

2.1 Two-Player Zero-Sum Games

Definition 2.1 (Zero-Sum Game). *A game is zero-sum if the payoff to one player is exactly the negative of the payoff to the other. If A_{ij} is the hider's payoff when the hider chooses location i and the seeker chooses location j , then the seeker's payoff for the same outcome is $-A_{ij}$.*

In this game:

- The **Hider** selects a location $i \in \{0, \dots, n-1\}$ to hide in.
- The **Seeker** independently selects a location j to search.
- If $i = j$ (the seeker finds the hider), the seeker wins; otherwise the hider wins.

2.2 Payoff Matrix Construction

Each location is assigned a *difficulty category* drawn uniformly at random from $\{1, 2, 3\}$, corresponding to **easy**, **neutral**, and **hard**. The payoff A_{ij} (hider's gain) is:

$$A_{ij} = \begin{cases} -1 & \text{if } i = j \text{ and category is } \mathbf{easy} \\ +2 & \text{if } i \neq j \text{ and category is } \mathbf{easy} \\ -1 & \text{if } i = j \text{ and category is } \mathbf{neutral} \\ +1 & \text{if } i \neq j \text{ and category is } \mathbf{neutral} \\ -3 & \text{if } i = j \text{ and category is } \mathbf{hard} \\ +1 & \text{if } i \neq j \text{ and category is } \mathbf{hard} \end{cases}$$

Intuitively:

- **Easy** locations are attractive to hide in (high reward if not found) but carry a small penalty if found.
- **Hard** locations carry a large penalty if found, reflecting the difficulty of escaping once discovered.
- The diagonal (caught) entry is always negative for the hider; all off-diagonal entries are positive (hider escapes).

3 Code Explanation

3.1 solver.py — HideAndSeekGame Class

3.1.1 __init__ and initialize_solver

```
1 @staticmethod
2 def initialize_solver(num_places):
3     places_criteria = [random.randint(1, 3) for _ in range(
4         num_places)]
5     category_names = {1: "easy", 2: "neutral", 3: "hard"}
6     places = [category_names[c] for c in places_criteria]
7
8     payoff_matrix = [[0]*num_places for _ in range(num_places)]
9     for i, k in enumerate(places_criteria):
10         if category_names[k] == "easy":
11             for j in range(num_places):
12                 payoff_matrix[i][j] = -1 if j == i else 2
13         elif category_names[k] == "neutral":
14             for j in range(num_places):
15                 payoff_matrix[i][j] = -1 if j == i else 1
16         else: # hard
17             for j in range(num_places):
18                 payoff_matrix[i][j] = -3 if j == i else 1
19     return payoff_matrix
```

Listing 1: Payoff matrix initialisation

Each location i is assigned a random category, and the full $n \times n$ payoff matrix A is built according to the rules in Section 2.2.

3.1.2 play_game — Single-Round Resolution

```
1 @staticmethod
2 def play_game(place, turn, num_places, payoff_matrix):
3     xc = random.randint(0, num_places - 1) # opponent's choice
4     hider_score = payoff_matrix[place][xc] if turn == "hider" \
5         else payoff_matrix[xc][place]
6     if place == xc: # caught
7         seeker_score = -hider_score
8         hider_score = 0
9     else:
10         seeker_score = 0
11
12     winner = ("seeker" if xc == place else "hider") if turn == "
13         seeker" \
14         else ("hider" if xc != place else "seeker")
15     ...
```

Listing 2: Single round play

Key design points:

1. The opponent plays uniformly at random (no strategy awareness).
2. The payoff lookup is role-aware: when the *human* is the hider, `payoff_matrix[place][xc]` applies directly (row = hider, column = seeker); when the human is the seeker, the indices are transposed.
3. Caught/not-caught zero-sum accounting ensures scores sum to zero (the seeker receives $-A_{ij}$ when $i = j$).

3.1.3 simulate_game

```

1 @staticmethod
2 def simulate_game(num_places, payoff_matrix, num_rounds=100,
3                   player_role="hider"):
4     player_score = computer_score = 0
5     rounds_won_player = rounds_won_computer = 0
6     for _ in range(num_rounds):
7         place = random.randint(0, num_places - 1)
8         result = HideAndSeekGame.play_game(place, player_role,
9                                             num_places,
10                                             payoff_matrix)
11
12         if result["winner"] == player_role:
13             rounds_won_player += 1
14             player_score += result["your_score"]
15         else:
16             rounds_won_computer += 1
17             computer_score += result["opponent_score"]
18     ...

```

Listing 3: Monte-Carlo simulation

Both the human player and the computer choose locations uniformly at random. This provides a *baseline* comparison against the LP-optimal strategy.

3.1.4 solve_payoff_matrix — LP via CPLEX

```

1 # Hider: maximise v
2 myproblem.objective.set_sense(myproblem.objective.sense.maximize)
3 x_names = [f"x{i}" for i in range(m)]
4 myproblem.variables.add(names=x_names, lb=[0.0]*m)
5 v_name = ["v"]
6 myproblem.variables.add(names=v_name, lb=[-cplex.infinity])
7
8 # sum(x_i) = 1
9 myproblem.linear_constraints.add(
10     lin_expr=[cplex.SparsePair(ind=x_names, val=[1.0]*m)],
11     senses=["E"], rhs=[1.0])
12
13 # For each seeker action j: sum_i A[i][j]*x_i >= v
14 for j in range(n):
15     myproblem.linear_constraints.add(
16         lin_expr=[cplex.SparsePair(

```

```

17         ind=x_names + v_name,
18         val=[payoff_matrix[i][j] for i in range(m)] + [-1.0])
19     ],
20     senses=["G"], rhs=[0.0])
21 myproblem.objective.set_linear("v", 1.0)
22 myproblem.solve()

```

Listing 4: Hider LP (maximisation)

```

1 # Seeker: minimise w
2 myproblem.objective.set_sense(myproblem.objective.sense.minimize)
3 y_names = [f"y{j}" for j in range(n)]
4 myproblem.variables.add(names=y_names, lb=[0.0]*n)
5 w_name = ["w"]
6 myproblem.variables.add(names=w_name, lb=[-cplex.infinity])
7
8 # sum(y_j) = 1
9 myproblem.linear_constraints.add(
10     lin_expr=[cplex.SparsePair(ind=y_names, val=[1.0]*n)],
11     senses=["E"], rhs=[1.0])
12
13 # For each hider action i: sum_j A[i][j]*y_j <= w
14 for i in range(m):
15     myproblem.linear_constraints.add(
16         lin_expr=[cplex.SparsePair(
17             ind=y_names + w_name,
18             val=[payoff_matrix[i][j] for j in range(n)] + [-1.0])
19         ],
20         senses=["L"], rhs=[0.0])
21 myproblem.objective.set_linear("w", 1.0)
22 myproblem.solve()

```

Listing 5: Seeker LP (minimisation)

Note how the seeker’s LP iterates over *rows* (hider’s actions) whereas the hider’s LP iterates over *columns* (seeker’s actions), correctly capturing the dual relationship.

3.2 main.py — Flask REST API

3.2.1 Endpoints

Route	Method	Purpose
/api/solve	POST	Initialise game; solve LP for optimal strategy
/api/play	POST	Execute one human-chosen round
/api/simulate	POST	Run simulation
/api/reset	POST	Clear all game state
/api/state	GET	Return current score summary

3.2.2 Global State Management

A module-level `game_state` dictionary persists the payoff matrix and running scores across requests. Key design decisions:

- **Matrix persistence:** `/api/solve` stores the matrix. Subsequent `/api/play` and `/api/simulate` calls reuse it, ensuring all rounds operate on the same game instance.
- **Fallback creation:** If the matrix is absent or the number of places changes, a new `HideAndSeekGame` is created and stored automatically.
- **Clean reset:** `/api/reset` sets the matrix to `None`, forcing a fresh game on the next `/api/solve`.

3.2.3 Role Propagation

Both `/api/solve` and `/api/simulate` accept a `player_role` field (`"hider"` or `"seeker"`). The computer adopts the opposite role. This ensures the LP is solved from the correct perspective and simulation scores are attributed correctly.